

Код на языке программирования Python

```
import cmath
import math
import numpy as np

try:
    print("Введите коэффициенты уравнения 4-ой степени через пробел:", end=" ")
    string = input() # строка с коэффициентами
    a_str, b_str, c_str, d_str, e_str = string.split(' ') # разделяем коэффициенты, но оставляем
их строками
    strings = [a_str, b_str, c_str, d_str, e_str] # массив коэффициентов в формате строк
    a, b, c, d, e = map(float, string.split(' ')) # получаем числовые коэффициенты

    # масштабируем коэффициенты (ищем такую степень 10, на которую можно умножить
все коэффициенты и получить целые
    # числа, тем самым увеличив точность и скорость выполнения программы)

    # смотрим числа с запятой (вещественные числа превращаем в целые, умножая на 10 **
(-max_count))
    max_count = 0
    for i in range(5):
        if strings[i].find('.') != -1:
            count = abs(strings[i].find('.') - len(strings[i])) - 1
            max_count = max(max_count, count)
    s = -max_count
    k_1 = 10 ** (-s) # масштабирующий коэффициент

    # получаем коэффициенты без погрешности (из чисел с запятой получаем числа без
запятой)
    a = round(a * k_1)
    b = round(b * k_1)
    c = round(c * k_1)
    d = round(d * k_1)
    e = round(e * k_1)

    # смотрим большие числа с нулями на конце
    while (a % 10 == 0) and (b % 10 == 0) and (c % 10 == 0) and (d % 10 == 0) and (e % 10 ==
0) and \
        (a != 0 or b != 0 or c != 0 or d != 0 or e != 0):
        a /= 10
        b /= 10
        c /= 10
        d /= 10
        e /= 10

    print("Масштабированные коэффициенты:", a, b, c, d, e)
```

```

if a == 0:
    if b == 0:
        if c == 0:
            if d == 0:
                if e == 0:
                    print("x - любое число")
                else:
                    print("Решений нет")
            else: # линейное уравнение
                x = -e / d
                print(f"Одно решение: {x}")
        else: # квадратное уравнение
            Discr = d ** 2 - 4 * c * e # считаем квадратный дискриминант
            if Discr == 0:
                x = -d / (2 * c)
                print(f"Одно решение: {x}")
            else:
                x_1 = (-d + cmath.sqrt(Discr)) / (2 * c)
                x_2 = (-d - cmath.sqrt(Discr)) / (2 * c)
                if Discr > 0:
                    print(f"Два решения: {x_1.real} {x_2.real}")
                else:
                    print(f"Два решения: {x_1} {x_2}")
    else: # кубическое уравнение (формулы Кардано)
        # необходимо свести изначальное уравнение вида  $bx^3 + cx^2 + dx + e = 0$  к
уравнению вида  $x^3 + px + q = 0$ 
        p = (3 * b * d - c ** 2) / (3 * b ** 2)
        q = (2 * c ** 3 - 9 * b * c * d + 27 * b ** 2 * e) / (27 * b ** 3)

        # перешли к уравнению вида  $x^3 + px + q = 0$ 
        QD = (p / 3) ** 3 + (q / 2) ** 2 # считаем кубический дискриминант
        QD = round(QD, 10) # округляем для повышения точности

        # сначала рассмотрим частные случаи, а потом будем искать корни через
кубический дискриминант
        if p == 0 and q == 0: # находим корни уравнения вида  $x^3 = 0$ 
            x_1 = 0 - c / (3 * b)
            print(f"Одно решение: {x_1}")
        elif q == 0 and p != 0: # находим корни уравнения вида  $x^3 + px = 0$ 
            x_1 = 0 - c / (3 * b)
            if p > 0:
                # два комплексных корня
                x_2 = cmath.sqrt(-p) - c / (3 * b)
                x_3 = -cmath.sqrt(-p) - c / (3 * b)
            else:
                # два вещественных корня
                x_2 = math.sqrt(-p) - c / (3 * b)
                x_3 = -math.sqrt(-p) - c / (3 * b)

```

```

print(f"Три решения: {x_1} {x_2} {x_3}")

elif p == 0 and q != 0: # находим корни уравнения вида  $x^3 + q = 0$ 
    x_1 = np.cbrt(-q) - c / (3 * b)
    print(f"Одно решение: {x_1}")

# p != 0 и q != 0
elif QD == 0: # получим два различных вещественных корня
    alfa_0 = np.cbrt((-q / 2))
    x_1 = 2 * alfa_0 - c / (3 * b)
    x_2 = -alfa_0 - c / (3 * b)
    print(f"Два решения: {x_1} {x_2}")

elif QD < 0: # 3 вещественных корня (могут быть кратными)
    theta = math.acos(-q / (2 * math.sqrt(-(p / 3) ** 3)))
    x_1 = 2 * math.sqrt(-p / 3) * math.cos(theta / 3) - c / (3 * b)
    x_2 = 2 * math.sqrt(-p / 3) * math.cos(theta / 3 + 2 * math.pi / 3) - c / (3 * b)
    x_3 = 2 * math.sqrt(-p / 3) * math.cos(theta / 3 - 2 * math.pi / 3) - c / (3 * b)

    if x_1 == x_2:
        print(f"Два решения: {x_1} {x_3}")
    elif x_2 == x_3 or x_1 == x_3:
        print(f"Два решения: {x_1} {x_2}")
    else:
        print(f"Три решения: {x_1} {x_2} {x_3}")

elif QD > 0 and p > 0: # 3 корня: один вещественный и два комплексных
    сопряженных
    beta_1 = math.atan(2 * math.sqrt((p / 3) ** 3) / q)
    while abs(beta_1) > math.pi / 2: # abs(beta_1) <= pi/2
        if beta_1 < 0:
            beta_1 += math.pi / 2
        else:
            beta_1 -= math.pi / 2
    alpha_1 = math.atan(np.cbrt(math.tan(beta_1 / 2)))
    while abs(alpha_1) > math.pi / 4: # abs(alpha_1) <= pi/4
        if alpha_1 < 0:
            alpha_1 += math.pi / 4
        else:
            alpha_1 -= math.pi / 4
    x_1 = -2 * math.sqrt(p / 3) / math.tan(2 * alpha_1) - c / (3 * b)
    x_2 = math.sqrt(p / 3) * (1 / math.tan(alpha_1 * 2) + cmath.sqrt(-3) * (1 / math.sin(2 *
alpha_1))) - c / (3 * b)
    x_3 = math.sqrt(p / 3) * (1 / math.tan(alpha_1 * 2) - cmath.sqrt(-3) * (1 / math.sin(2 *
alpha_1))) - c / (3 * b)
    print(f"Три решения: {x_1} {x_2} {x_3}")

```

```

else: # QD > 0 и p < 0, получим 3 корня: один вещественный и два комплексных
сопряженных
    # (этот случай считается по другим формулам)
    beta_2 = math.asin(2 * math.sqrt(-(p / 3) ** 3) / q)
    while abs(beta_2) > math.pi / 2: # abs(beta_2) <= pi/2
        if beta_2 < 0:
            beta_2 += math.pi / 2
        else:
            beta_2 -= math.pi / 2
    alpha_2 = math.atan(np.cbrt(math.tan(beta_2 / 2))) # abs(alpha_2) <= pi/4
    while abs(alpha_2) > math.pi / 4:
        if alpha_2 < 0:
            alpha_2 += math.pi / 4
        else:
            alpha_2 -= math.pi / 4
    x_1 = -2 * math.sqrt(-p / 3) * (1 / math.sin(2 * alpha_2)) - c / (3 * b)
    x_2 = math.sqrt(-p / 3) * (1 / math.sin(2 * alpha_2) + cmath.sqrt(-3) / math.tan(2 *
alpha_2)) - c / (3 * b)
    x_3 = math.sqrt(-p / 3) * (1 / math.sin(2 * alpha_2) - cmath.sqrt(-3) / math.tan(2 *
alpha_2)) - c / (3 * b)
    print(f"Три решения: {x_1} {x_2} {x_3}")

else: # уравнение 4-ой степени (метод хорд)
    try:
        print("Введите через пробел границы отрезка, на котором будут находиться корни
уравнения, и их точность:", end = " ")
        string_1 = input()
        start_str, end_str, epsilon_str = string_1.split(' ')
        start, end, epsilon = map(float, string_1.split(' '))

        if epsilon <= 0:
            print("Ошибка: точность должна быть положительной!")
        else:
            # масштабируем границы отрезка для увеличения точности поиска корней
            max_count_1 = 0
            strings_1 = [start_str, end_str, epsilon_str]
            for i in range(2):
                if strings_1[i].find('.') != -1:
                    count_1 = abs(strings_1[i].find('.') - len(strings_1[i])) - 1
                    max_count_1 = max(max_count_1, count_1)
            k_2 = 10 ** max_count_1
            start = round(start * k_2)
            end = round(end * k_2)

            if start < end:
                x = []
                k = 0
                steps = [] # массив подотрезков, на которых будут искаться корни

```

```

start *= 1000
end *= 1000
step = start
while step < end: # заполняем массив подотрезков (идти будем по общему
отрезку с шагом 0,001)
    step = step + 1
    steps.append(step / 1000 / k_2)

alfa = steps[0] # нижняя граница рассматриваемого подотрезка

# проверяем, является ли нижняя граница отрезка, вводимого пользователем,
решением уравнения
if (a * (start / 1000) ** 4 + b * (start / 1000) ** 3 + c * (start / 1000) ** 2 + d * (start
/ 1000) + e) == 0:
    x.append((start / 1000))
    k += 1
# идем по отрезку с шагом 0,001 и ищем корни
for i in range(1, len(steps)):
    beta = steps[i] # верхняя граница рассматриваемого подотрезка
    if (a * alfa ** 4 + b * alfa ** 3 + c * alfa ** 2 + d * alfa + e) * (a * beta ** 4 + b *
beta ** 3 + c * beta ** 2 + d * beta + e) <= 0:
        # новые границы рассматриваемого подотрезка, которые будут сдвигаться
и искать корень
        alfa_1 = alfa
        beta_1 = beta
        # предполагаемый корень
        t = (alfa_1 * (
            a * beta_1 ** 4 + b * beta_1 ** 3 + c * beta_1 ** 2 + d * beta_1 + e) -
beta_1 * (
            a * alfa_1 ** 4 + b * alfa_1 ** 3 + c * alfa_1 ** 2 + d * alfa_1 + e))
        / (
            (a * beta_1 ** 4 + b * beta_1 ** 3 + c * beta_1 ** 2 + d * beta_1 + e)
- (
            a * alfa_1 ** 4 + b * alfa_1 ** 3 + c * alfa_1 ** 2 + d * alfa_1 +
e))
        # доводим точность корня до допустимой (epsilon), сужая границы
отрезка [alfa_1, beta_1]
        while (a * t ** 4 + b * t ** 3 + c * t ** 2 + d * t + e) >= epsilon:
            if (a * t ** 4 + b * t ** 3 + c * t ** 2 + d * t + e) * (a * alfa_1 ** 4 + b *
alfa_1 ** 3 + c * alfa_1 ** 2 + d * alfa_1 + e) < 0:
                beta_1 = t
            else:
                alfa_1 = t
        # заново пересчитываем корень
        t = (alfa_1 * (
            a * beta_1 ** 4 + b * beta_1 ** 3 + c * beta_1 ** 2 + d * beta_1 + e) -
beta_1 * (

```

```

        a * alfa_1 ** 4 + b * alfa_1 ** 3 + c * alfa_1 ** 2 + d * alfa_1 + e))
/ (
        (a * beta_1 ** 4 + b * beta_1 ** 3 + c * beta_1 ** 2 + d * beta_1 + e)
- (
        a * alfa_1 ** 4 + b * alfa_1 ** 3 + c * alfa_1 ** 2 + d * alfa_1 + e))
    if t not in x: # добавляем корень в массив корней, если его там еще нет
        x.append(t)
        k += 1
    alfa = beta # переходим на следующий шаг (рассматриваем следующий
подотрезок)
    if k == 0:
        print("Решений нет")
    elif k == 1:
        print(f'Одно решение: {x[0]}')
    elif k == 2:
        print(f'Два решения: {x[0]} {x[1]}')
    elif k == 3:
        print(f'Три решения: {x[0]} {x[1]} {x[2]}')
    else:
        print(f'Четыре решения: {x[0]} {x[1]} {x[2]} {x[3]}')
    print("Ошибка: начало отрезка должно быть меньше его конца!")
except ValueError:
    print("Ошибка: некорректный ввод данных!")
except ValueError:
    print("Ошибка: некорректный ввод данных!")

```